

Macchine di Turing e Decidibilità

Marco Galletti

14 aprile 2026

Indice

1	Esercizio 1	1
2	Esercizio 2	3
3	Esercizio 3	5
4	Esercizio 4	5
5	Esercizio 5	6
6	Esercizio 6	7
7	Esercizio 7	13
8	Esercizio 8	14
9	Esercizio 9	15
10	Esercizio 10	17

1 Esercizio 1

1. Calcolo di $n + 1$ su input binario

La MdT usa tre soli stati:

- q_0 (iniziale): posizionata sull'ultimo bit del numero
 - su 0: scrive 1 e va in q_{acc} (fine);
 - su 1: scrive 0 e passa in q_{carry} ;
 - su $\sqcup$: scrive 1 e va in q_{acc} (estende il numero).
- q_{carry} (gestione riporto)
 - su 1: scrive 0, resta in q_{carry} e si sposta a sinistra;

- su 0 o $\sqcup$: scrive 1 e va in q_{acc} .
- q_{acc} (accettazione e arresto): la macchina si ferma con il risultato corretto sul nastro.

2. Verifica se 1^n è multiplo di 3

La MdT usa cinque stati:

- q_0 (iniziale, resto 0)
 - su 1: vai in q_1 ;
 - su $\sqcup$: vai in q_{acc} .
- q_1 (resto 1)
 - su 1: vai in q_2 ;
 - su $\sqcup$: vai in q_{rej} .
- q_2 (resto 2)
 - su 1: vai in q_0 ;
 - su $\sqcup$: vai in q_{rej} .
- q_{acc} : accetta.
- q_{rej} : rifiuta.

3. L'input appartiene all'insieme $A = \{0^n 1^n : n \geq 0\}$

La MdT usa sei stati:

- q_{start} (iniziale)
 - cerca il primo 0 non marcato, lo sostituisce con X e va in q_{find1} ;
 - se legge $\sqcup$ (niente 0 residui), va in q_{check} .
- q_{find1}
 - cerca il primo 1 non marcato, lo sostituisce con Y e va in q_{return} ;
 - se arriva a $\sqcup$ senza trovarne, va in q_{rej} .
- q_{return}
 - torna all'inizio del nastro e va in q_{start} .
- q_{check}
 - scansiona: se non trova 1 non marcati, va in q_{acc} ; altrimenti in q_{rej} .
- q_{acc} : accetta.
- q_{rej} : rifiuta.

4. Moltiplicazione $1^n * 1^m \mapsto 1^{n \times m}$

La MdT usa cinque stati:

- q_{start} (iniziale)
 - cerca il primo 1 prima di *, lo marca con X e va in q_{copy} ;
 - se legge * senza trovarne, va in q_{cleanup} .
- q_{copy}
 - sposta oltre *, quindi per ogni 1 nella seconda porzione scrive un 1 in coda;
 - al termine torna in q_{return} .
- q_{return}
 - torna all’inizio del nastro (oltre le X) e va in q_{start} .
- q_{cleanup}
 - cancella tutte le X e la *, ricomponendo il blocco di 1, poi va in q_{acc} .
- q_{acc} : accetta.

2 Esercizio 2

1. Riconoscere il linguaggio $L = \{ ww : w \in \{0, 1\}^* \}$

Idea generale. Data una stringa x , la MdT deve verificare se esiste una suddivisione $x = ww$ (stesso w ripetuto due volte). A livello intuitivo, la macchina può:

1. Cercare di “marcare” corrispondenze tra la prima metà e la seconda metà.
2. A ogni passo, seleziona il primo simbolo non ancora marcato nella parte iniziale di x , lo cerca nella parte finale e, trovata la corrispondenza, marca entrambe le posizioni.
3. Deve poi garantire che il numero di simboli, e l’ordine, combaci per tutta la stringa.

Un approccio più semplice (ma costoso) è:

- Stimare la lunghezza n di x . Se n è dispari, rifiuta subito.
- Sapendo n (ora pari), individua la “metà” $\frac{n}{2}$.
- Controlla, per $i = 1$ fino a $\frac{n}{2}$, che il (i) -esimo simbolo coincida con il $(\frac{n}{2} + i)$ -esimo simbolo. Se per tutte le posizioni c’è match, accetta; altrimenti rifiuta.

Dettagli implementativi

- La MdT può contare la lunghezza di x spostandosi a destra e tornando a sinistra più volte (con un contatore di passaggi).
- Una volta nota la metà, esegue $i = 1$ fino a $\frac{n}{2}$: si posiziona sul simbolo i e poi su $i + \frac{n}{2}$, controllando se coincidono. In caso negativo, rifiuta.
- Se tutti i confronti vanno bene, accetta.

1. Riconoscere il linguaggio $L = \{ww : w \in \{0,1\}^*\}$

La MdT usa i seguenti stati:

- q_{start}
 - legge 0: scrive X , va in q_{search0} ;
 - legge 1: scrive X , va in q_{search1} ;
 - legge $\sqcup$: va in q_{acc} .
- q_{search0}
 - scorre a destra cercando un 0 non marcato, scrive Y e va in q_{return} ;
 - se incontra $\sqcup$ senza aver trovato 0, va in q_{rej} .
- q_{search1}
 - analogo a q_{search0} ma cerca un 1.
- q_{return} torna a sinistra fino al primo X o Y , poi si sposta oltre (primo simbolo non marcato) e torna in q_{start} .
- q_{acc} : accetta.
- q_{rej} : rifiuta.

2. Decidere i palindromi di forma ww^r (w^r è la stringa w invertita)

La MdT usa gli stessi stati di marcatura estremi:

- q_{start}
 - legge 0: scrive X , va in q_{search0} ;
 - legge 1: scrive X , va in q_{search1} ;
 - legge $\sqcup$: va in q_{acc} .
- q_{search0}
 - scorre a destra fino all'ultimo simbolo non marcato; se è 0 scrive Y e va in q_{return} , altrimenti va in q_{rej} .

- q_{search1}
 - analogo a q_{search0} ma verifica un 1.
- q_{return} torna a sinistra fino al primo X o Y , poi si sposta oltre e va in q_{start} .
- q_{acc} : accetta.
- q_{rej} : rifiuta.

3 Esercizio 3

1. Busy Beaver con 3 stati operativi $(s_0, s_1, s_2) + \text{Halt}$

- Alfabeto del nastro: $\Gamma = \{0, 1, \sqcup\}$, con blank $\sqcup$.
- Stati: $Q = \{s_0, s_1, s_2, \text{Halt}\}$, con s_0 iniziale e Halt di arresto.

La funzione di transizione $\delta: Q \times \Gamma \rightarrow \Gamma \times \{L, R\} \times Q$ è

	legge 0	legge 1
s_0	$(1, R, s_1)$	$(1, R, \text{Halt})$
s_1	$(0, R, s_2)$	$(1, R, s_1)$
s_2	$(1, L, s_2)$	$(1, L, s_0)$
Halt	—	—

Partendo da nastro vuoto, questa macchina scrive esattamente 6 simboli 1 consecutivi e poi si arresta.

2. Busy Beaver con 4 stati operativi $(s_0, s_1, s_2, s_3) + \text{Halt}$

- Alfabeto: $\Gamma = \{0, 1, \sqcup\}$.
- Stati: $Q = \{s_0, s_1, s_2, s_3, \text{Halt}\}$.

	legge 0	legge 1
s_0	$(1, R, s_1)$	$(1, L, s_1)$
s_1	$(1, L, s_0)$	$(0, L, s_2)$
s_2	$(1, R, \text{Halt})$	$(1, L, s_3)$
s_3	$(1, R, s_3)$	$(0, R, s_0)$
Halt	—	—

Questa macchina stampa esattamente 13 simboli 1 consecutivi e poi si arresta.

4 Esercizio 4

Soluzione

Sia $A \leq B$ e supponiamo che B sia decidibile. Vogliamo mostrare che allora anche A è decidibile.

Idea della dimostrazione. Poiché $A \leq B$, esiste una funzione calcolabile f tale che, per ogni stringa w ,

$$w \in A \iff f(w) \in B.$$

Essendo B decidibile, esiste una macchina di Turing (o un algoritmo) M_B che, data in input una stringa x , stabilisce *in tempo finito* se $x \in B$ oppure no.

Costruzione di una macchina M_A che decide A .

1. Dato in input w , la macchina M_A calcola la funzione $f(w)$.
2. Esegue la macchina M_B su $f(w)$.
3. Se M_B accetta, allora $w \in A$ (poiché $f(w) \in B$), e quindi M_A accetta.
4. Se M_B rifiuta, allora $w \notin A$, e quindi M_A rifiuta.

Conclusione. La macchina M_A si ferma sempre (perché M_B è decidibile) e risponde correttamente in merito all'appartenenza di w ad A . Pertanto A risulta decidibile.

5 Esercizio 5

Soluzione

Dobbiamo dimostrare che la relazione di riducibilità tra linguaggi è *transitiva*. In particolare, se $A \leq B$ e $B \leq C$, allora $A \leq C$.

Dimostrazione. Poiché $A \leq B$, esiste una funzione calcolabile f tale che, per ogni stringa w ,

$$w \in A \iff f(w) \in B.$$

Inoltre, poiché $B \leq C$, esiste una funzione calcolabile g tale che, per ogni stringa x ,

$$x \in B \iff g(x) \in C.$$

Consideriamo la funzione composizione $h = g \circ f$, definita da

$$h(w) = g(f(w)).$$

La funzione h è calcolabile (poiché f e g lo sono).

Verifichiamo la riduzione. Per ogni w :

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C \iff h(w) \in C.$$

Dunque h è una funzione di riduzione da A a C , e ciò dimostra che $A \leq C$.

Conclusione. Abbiamo così mostrato la *transitività* della riducibilità tra linguaggi:

$$A \leq B \text{ e } B \leq C \implies A \leq C.$$

6 Esercizio 6

Sia $M = (Q, \Sigma, \Gamma, \delta, \vdash, \sqcup, s, a, r)$ una generica macchina di Turing.

Di seguito, per ciascun linguaggio tra quelli elencati, determiniamo se è decidibile o indecidibile, fornendone una motivazione o una breve dimostrazione.

$A = \{\langle M \rangle \mid M \text{ ha almeno 1234 stati}\}$

Decisione. Questo è *decidibile*.

- In input riceviamo una stringa che codifica una macchina di Turing, denotata $\langle M \rangle$.
- Verifichiamo se la stringa $\langle M \rangle$ è una codifica valida di una macchina di Turing; se non lo è, rifiutiamo.
- Altrimenti, dalla descrizione di M estraiamo l'insieme dei suoi stati e ne contiamo il numero, diciamo n .
- Confrontiamo n con 1234:
 - Se $n \geq 1234$, accettiamo.
 - Altrimenti, rifiutiamo.
- Tutti questi passi (parsing della codifica, conteggio e confronto) terminano in tempo finito deterministico, quindi l'algoritmo decide la proprietà.

$B = \{\langle M \rangle \mid M \text{ esegue almeno 1234 passi su input } 0\}$

Decisione. Questo è *decidibile*.

- In input riceviamo una stringa che codifica una macchina di Turing, denotata $\langle M \rangle$.
- Verifichiamo se $\langle M \rangle$ è una codifica valida di una macchina di Turing; se non lo è, rifiutiamo.
- Altrimenti, simuliamo M su input 0 per esattamente 1234 passi.
- Se durante la simulazione M si arresta in un numero di passi $t < 1234$, rifiutiamo (poiché non ha eseguito 1234 passi).
- Se invece la simulazione raggiunge il passo 1234 senza arrestarsi, accettiamo (allora M esegue almeno 1234 passi, sia che poi si arresti o che continui a girare).
- La simulazione di un numero finito di passi è un'operazione che termina sempre in tempo deterministico, perciò questo algoritmo decide B .

$C = \{\langle M \rangle \mid \exists x \text{ } M \text{ esegue almeno 1234 passi su input } x\}$

Decisione. Questo è *decidibile*.

- In input riceviamo una stringa $\langle M \rangle$ che codifica una macchina di Turing M .
- Verifichiamo se $\langle M \rangle$ è una codifica valida; se non lo è, rifiutiamo.
- Sia $k = 1234$. Osserviamo che in k passi la testa di M può spostarsi di al più k celle a destra o a sinistra dal punto di partenza.
- Di conseguenza, per ogni possibile input x , il comportamento di M nei primi k passi dipende soltanto dal contenuto di x nelle posizioni $-k, -k+1, \dots, 0, \dots, k-1, k$ (e il resto del nastro è vuoto).
- Costruiamo dunque l'insieme finito

$$S = \Sigma^{2k+1}$$

di tutte le possibili stringhe di lunghezza esattamente $2k+1$ sull'alfabeto di input di M , interpretate come configurazioni iniziali di quelle $2k+1$ celle centrali (il resto è blank).

- Per ciascuna $x \in S$, simuliamo M su x per al più k passi:
 - Se esiste almeno un $x \in S$ per cui la simulazione non si arresta entro k passi, allora M esegue almeno k passi su quell'input e quindi appartiene a C : *accettiamo*.
 - Se invece per ogni $x \in S$ la simulazione si arresta in meno di k passi, allora su *qualsiasi* input (anche più lungo) M si comporterà allo stesso modo nei primi k passi e si arresta comunque prima di k : *rifiutiamo*.
- Poiché S è finito e ogni simulazione di k passi termina in tempo deterministico, l'algoritmo decide C .

$D = \{\langle M \rangle \mid \forall x \text{ } M \text{ esegue almeno 1234 passi su input } x\}$

Decisione. Questo è *decidibile*.

- In input riceviamo una stringa $\langle M \rangle$ che codifica una macchina di Turing M .
- Verifichiamo se $\langle M \rangle$ è una codifica valida; se non lo è, rifiutiamo.
- Poniamo $k = 1234$. Notiamo che per determinare se M si arresta in meno di k passi su un qualunque input, basta considerare la “finestra” di celle $[-k, \dots, +k]$ attorno alla testina iniziale, di ampiezza $2k+1$, come nel caso precedente.
- Definiamo l'insieme finito

$$S = \Sigma^{2k+1}$$

di tutte le possibili configurazioni iniziali su quelle $2k+1$ celle (il resto del nastro è vuoto).

- Per ogni configurazione $s \in S$:
 - Simuliamo M su s per al più $k-1$ passi.

- Se in qualche simulazione M si arresta in un numero di passi $t < k$, significa che esiste un input (anche più lungo) che coincide con s nella finestra e su cui M si ferma prima di k : in tal caso $\langle M \rangle \notin D$, quindi rifiutiamo.
- Se per tutte le $s \in S$ la simulazione raggiunge il passo k senza arrestarsi, allora su ogni input x (il cui comportamento nei primi k passi è determinato dalla stessa finestra) M farà almeno k passi: accettiamo.
- Poiché S è finito e ogni simulazione si interrompe entro un numero finito di passi, l'algoritmo termina sempre e decide D .

$E = \{ \langle M \rangle \mid \exists x \text{ tale che la testina di } M, \text{ eseguito su } x, \text{ visita la cella } +1234 \}$

Decisione. Questo è *decidibile*.

- Dato l'input $\langle M \rangle$, verifichiamo che sia una codifica valida; se non lo è, rifiutiamo.
- Fissiamo $k = 1234$ e sia Σ l'alfabeto di input, Γ quello di nastro, Q l'insieme degli stati di M .
- **Osservazione chiave.** Prima di superare la cella $+k$ la testina non legge celle con indice $> k$; dunque il comportamento nei primi $k+1$ celle dipende solo dai simboli di un input lungo al massimo k .
- **Insieme finito di input da esaminare.** Enumeriamo tutte le stringhe $x \in \Sigma^{\leq k}$; sono in numero finito (al più $\sum_{i=0}^k |\Sigma|^i$).
- **Test su un singolo x .** Consideriamo solo le celle $0, \dots, k$. Una *configurazione limitata* è la terna (q, p, t) con $q \in Q$, $p \in \{0, \dots, k\}$ e $t \in \Gamma^{k+1}$. Il numero di tali configurazioni è

$$T = |Q| (k + 1) |\Gamma|^{k+1}.$$

Simuliamo M su x conservando la storia delle configurazioni limitate:

1. Se la testina si muove da k a $k + 1$ accettiamo (esiste un input “buono”).
 2. Se in $T + 1$ passi ciò non accade, per il principio dei cassetti si ripete una configurazione limitata; d'ora in poi M resterà in un ciclo interno a $[0, k]$. Passiamo quindi al prossimo x .
- Poiché l'elenco di input è finito e ogni test richiede al più $T + 1$ passi simulati, l'algoritmo termina sempre:
 - se almeno un test attiva il punto (1) sopra, accettiamo;
 - altrimenti, dopo aver esaurito tutti gli x , rifiutiamo.

$$F = \{\langle M \rangle \mid M \text{ accetta la stringa vuota } \varepsilon\}$$

Decisione. Questo è *non decidibile*.

- **Problema di partenza (HP).**

$$\text{HP} = \{\langle M, x \rangle \mid M \text{ si arresta su } x\}$$

è indecidibile.

- **Costruzione della riduzione.** Definiamo la funzione calcolabile

$$f: \langle M, x \rangle \mapsto \langle N \rangle$$

dove la macchina N su input ε :

1. simula M su x ;
2. se la simulazione di M si arresta (accetta o rigetta), N accetta ε ;
3. altrimenti rimane in loop.

- **Correttezza della riduzione.**

- Se $\langle M, x \rangle \in \text{HP}$ (cioè M si arresta su x), la simulazione termina $\Rightarrow N$ accetta $\varepsilon \Rightarrow \langle N \rangle \in F$.
- Se $\langle M, x \rangle \notin \text{HP}$, M non si arresta su $x \Rightarrow N$ rimane in loop su $\varepsilon \Rightarrow \langle N \rangle \notin F$.

- **Conclusione.** Poiché esiste una riduzione effettiva $\text{HP} \leq_m F$ e HP è indecidibile, anche F è indecidibile.

$$G = \{\langle M \rangle \mid M \text{ accetta almeno una stringa}\}$$

Decisione. Questo è *non decidibile*.

- **Problema di partenza (HP).**

$$\text{HP} = \{\langle M, x \rangle \mid M \text{ si arresta su } x\}$$

è indecidibile.

- **Riduzione.** Data $\langle M, x \rangle$, costruiamo una macchina N che su **qualsiasi** input w :

1. Simula M su x .
2. Se la simulazione si arresta, N accetta w ; altrimenti rimane in loop.

La funzione $\langle M, x \rangle \mapsto \langle N \rangle$ è calcolabile.

- **Correttezza.**

- Se $\langle M, x \rangle \in \text{HP}$, M si arresta su $x \Rightarrow N$ accetta ogni $w \Rightarrow N$ accetta almeno una stringa $\Rightarrow \langle N \rangle \in G$.
- Se $\langle M, x \rangle \notin \text{HP}$, M non si arresta su $x \Rightarrow N$ non accetta mai $\Rightarrow \langle N \rangle \notin G$.

- **Conclusione.** Poiché esiste una riduzione effettiva $\text{HP} \leq_m G$ e HP è indecidibile, anche G è indecidibile.

$$H = \{\langle M \rangle \mid (M) \text{ accetta un insieme finito di input}\}$$

Decisione. Questo è *non decidibile*.

- **Problema di partenza (HP).**

$$\text{HP} = \{\langle M, x \rangle \mid M \text{ si arresta su } x\}$$

è indecidibile.

- **Costruzione della riduzione.** Data $\langle M, x \rangle$, costruiamo la macchina N che su input w :

1. Simula M su x .
2. Se la simulazione si arresta, N accetta w ; altrimenti rimane in loop.

La funzione $f: \langle M, x \rangle \mapsto \langle N \rangle$ è calcolabile.

- **Correttezza.**

- Se $\langle M, x \rangle \in \text{HP}$, cioè M si arresta su x , allora N accetta ogni w , perciò $|L(N)| = \infty$ e $\langle N \rangle \in H$.
- Se $\langle M, x \rangle \notin \text{HP}$, M non si arresta su x , allora N non accetta mai alcun w , perciò $|L(N)| = 0$ e $\langle N \rangle \in H$.

- **Conclusione.** Poiché esiste una riduzione effettiva $\text{HP} \leq_m \overline{H}$ e HP è indecidibile, segue che anche H non è decidibile.

$$I = \{\langle M \rangle \mid M \text{ accetta esattamente l'insieme dei numeri primi}\}$$

Decisione. Questo è *non decidibile*.

- **Problema di partenza (HP).**

$$\text{HP} = \{\langle A, x \rangle \mid A \text{ si arresta su } x\}$$

è indecidibile.

- **Definizione della riduzione.** Dato $\langle A, x \rangle$, costruiamo efficacemente una macchina N tale che:

1. Su qualunque input w , N simula A su x .
2. Se la simulazione si arresta, N esegue un test di primalità su w :

$$\begin{cases} \text{accetta,} & \text{se } w \text{ codifica un numero primo,} \\ \text{rigetta,} & \text{altrimenti.} \end{cases}$$

3. Se la simulazione non si arresta mai, N rimane in loop.

Questa trasformazione $\langle A, x \rangle \mapsto \langle N \rangle$ è calcolabile.

- **Correttezza.**

- Se $\langle A, x \rangle \in \text{HP}$ (ossia A si arresta su x), allora per ogni w la simulazione termina e N decide correttamente il test di primalità. Quindi

$$L(N) = \{\text{numeri primi}\} \implies \langle N \rangle \in I.$$

- Se $\langle A, x \rangle \notin \text{HP}$, A non si arresta su x , dunque N non raggiunge mai il test e non accetta alcun w . Quindi

$$L(N) = \emptyset \implies \langle N \rangle \notin I.$$

- **Conclusione.** Poiché esiste una riduzione effettiva $\text{HP} \leq_m I$ e HP è indecidibile, anche I è indecidibile.

$J = \{\langle M \rangle \mid \exists x \text{ tale che } M \text{ stampa il simbolo } \gamma\}$

Decisione. Questo è *non decidibile*.

- **Problema di partenza (HP).**

$$\text{HP} = \{\langle A, x \rangle \mid A \text{ si arresta su } x\}$$

è indecidibile.

- **Costruzione della riduzione.** Da $\langle A, x \rangle$ definiamo una macchina N che su **qualsiasi** input w :

1. Simula A su x .
2. Se la simulazione si arresta, N scrive il simbolo γ sulla cella corrente e si arresta.
3. Altrimenti rimane in loop.

La funzione $f: \langle A, x \rangle \mapsto \langle N \rangle$ è calcolabile.

- **Correttezza.**

- Se $\langle A, x \rangle \in \text{HP}$, ovvero A si arresta su x , allora su ogni input w la simulazione termina e N scrive γ . Quindi $\langle N \rangle \in J$.
- Se $\langle A, x \rangle \notin \text{HP}$, A non si arresta su x , allora N non raggiunge mai l'istruzione di scrittura e su nessun w stampa γ . Quindi $\langle N \rangle \notin J$.

- **Conclusione.** Poiché esiste una riduzione many-one $\text{HP} \leq_m J$ e HP è indecidibile, anche J è indecidibile.

$K = \{\langle M \rangle \mid \exists x \text{ tale che } M, \text{ eseguito su } x, \text{ entra nello stato } q\}$

Decisione. Questo è *non decidibile* (ma *ricorsivamente enumerabile*).

- **Problema di partenza (HP).**

$$\text{HP} = \{\langle A, x \rangle \mid A \text{ si arresta su } x\}$$

è indecidibile.

- **Costruzione della riduzione.** Data $\langle A, x \rangle$, costruiamo meccanicamente una macchina N che su **qualsiasi** input w :

1. Simula A su x .
2. Se la simulazione si arresta, N esegue una transizione nello stato q (poi può fermarsi o loopare).
3. Altrimenti rimane in loop.

La funzione $f: \langle A, x \rangle \mapsto \langle N \rangle$ è calcolabile.

- **Correttezza.**

- Se $\langle A, x \rangle \in \text{HP}$, cioè A si arresta su x , allora per ogni w la simulazione termina e N entra nello stato $q \Rightarrow \langle N \rangle \in K$.
- Se $\langle A, x \rangle \notin \text{HP}$, A non si arresta su x , quindi N non raggiunge mai q su nessun $w \Rightarrow \langle N \rangle \notin K$.

- **Conclusione.** Poiché esiste una riduzione many-one $\text{HP} \leq_m K$ e HP è indecidibile, K è indecidibile. Inoltre K è ricorsivamente enumerabile: basta enumerare tutti i pair (x, t) e simulare $M(x)$ per t passi, accettando se in uno di essi si entra in q .

7 Esercizio 7

Denotiamo con M_i la i -esima Macchina di Turing in un'enumerazione effettiva e definiamo

$$f(i) = \begin{cases} \text{numero di passi compiuti da } M_i \text{ su input vuoto,} & \text{se } M_i \text{ si arresta,} \\ 0, & \text{altrimenti,} \end{cases}$$

e

$$g(j) = \max_{i \leq j} f(i).$$

Mostriamo che g non è calcolabile.

Dimostrazione. Supponiamo, per assurdo, che esista una Macchina di Turing G che calcola

$$g(j) = \max_{i \leq j} f(i)$$

in un numero finito di passi su ogni input j .

Costruiamo allora una nuova MdT D che, ricevuto in input una stringa x ,

1. esegue G su x ottenendo $T = g(x)$;
2. compie esattamente $T + 1$ mosse “a vuoto”;
3. si arresta.

Sia d l'indice (nell'enumerazione effettiva di tutte le MdT) di questa macchina D . Allora, dato che $f(i)$ è definita come “numero di passi che M_i compie su input vuoto, o 0 se non si arresta”,

$$f(d) = \text{numero di passi di } D(d) = g(d) + 1.$$

Ma per definizione

$$g(d) = \max_{i \leq d} f(i) \geq f(d) = g(d) + 1,$$

da cui $g(d) \geq g(d) + 1$, assurdo.

Quindi non può esistere alcuna MdT che calcoli g , e g non è calcolabile. $\square$

8 Esercizio 8

1. Data una MdT M e una stringa x , determinare se M stamperà mai il simbolo $\#$ sul nastro

Definiamo il linguaggio

$$\text{PRINT}\# = \{ \langle M, x \rangle : M \text{ stampa il simbolo } \# \text{ sul nastro durante la computazione su input } x \}.$$

Teorema 8.1. $\text{PRINT}\#$ non è decidibile.

Dimostrazione. Supponiamo, per assurdo, che esista una macchina di Turing D che decidesse $\text{PRINT}\#$, ossia che su input $\langle M, x \rangle$ si arresti e accetti se e solo se M stampa almeno una volta il simbolo $\#$ durante la computazione su x .

Richiameremo il problema dell'Halting

$$\text{HALT} = \{ \langle M, x \rangle : M \text{ si arresta su } x \},$$

sapendo che HALT è indecidibile.

Costruzione della riduzione. Da un'istanza $\langle M, x \rangle$ di HALT costruiamo la macchina

$$M' = \text{“su qualunque input } y \text{ (anche vuoto): } \begin{cases} \text{simula } M \text{ su } x; \\ \text{se } M(x) \text{ si arresta, scrivi } \# \text{ e arrestati;} \\ \text{altrimenti continua a non terminare.} \end{cases}$$

Osserviamo che

$$M(x) \text{ si arresta} \iff M' \text{ stampa } \# \text{ (su } y).$$

Ora, se D decidesse $\text{PRINT}\#$, basterebbe costruire l'istanza $\langle M', y \rangle$ (per esempio $y = \varepsilon$) e lanciare D :

$$D(\langle M', \varepsilon \rangle) \text{ accetta} \iff M' \text{ stampa } \# \iff M \text{ si arresta su } x.$$

Così D risolverebbe il problema dell'Halting, contraddicendo la sua indecidibilità.

Conclusioni. Non può esistere alcuna MdT D che decida $\text{PRINT}\#$. Pertanto $\text{PRINT}\#$ non è decidibile. $\square$

2. Data una MdT M , stabilire se esistono *infinite* MdT equivalenti a M

Soluzione. Questo problema è *decidibile*, e anzi la risposta è sempre Sì, indipendentemente da M . Infatti, se M riconosce un certo linguaggio L , possiamo costruire infinite varianti di M (ad esempio aggiungendo stati o transizioni *inutili* che non alterino il comportamento effettivo) che riconoscono esattamente lo stesso linguaggio L . Ciò mostra che, data qualsiasi macchina M , è banale esibire infinite macchine fra loro equivalenti a M . Poiché la risposta è sempre “vero”, il problema risulta banalmente decidibile.

9 Esercizio 9

1. Esistenza di un input che porta la MdT M nello stato q

Problema di raggiungibilità di stato

Definiamo il linguaggio

$$\text{REACH} = \{ \langle M, q \rangle : \exists x \text{ tale che, simulando } M \text{ su } x, M \text{ entra nello stato } q \}.$$

Teorema 9.1. REACH non è decidibile.

Dimostrazione. Faremo una riduzione dal problema dell’Halting

$$\text{HALT} = \{ \langle M, x \rangle : M \text{ si arresta su } x \},$$

sapendo che HALT è indecidibile.

Costruzione della riduzione. Data un’istanza $\langle M, x \rangle$ di HALT, definiamo una nuova MdT M' come segue:

$$M'(y) = \begin{cases} \text{ignora } y, \\ \text{simula } M \text{ su } x, \\ \text{se } M(x) \text{ si arresta, passa allo stato } q^*, \\ \text{poi si arresta.} \end{cases}$$

Qui q^* è uno stato nuovo che abbiamo aggiunto a M' .

Allora:

$$\langle M, x \rangle \in \text{HALT} \iff \langle M', q^* \rangle \in \text{REACH}.$$

Infatti $M(x)$ si arresta se e solo se M' (su qualunque input y) arriva nello stato q^* .

Se esistesse una MdT D che decidesse REACH, basterebbe costruire $\langle M', q^* \rangle$ e lanciare D :

$$D(\langle M', q^* \rangle) \text{ accetta} \iff \langle M', q^* \rangle \in \text{REACH} \iff \langle M, x \rangle \in \text{HALT},$$

così D risolverebbe HALT, assurdo.

Conclusione. Non può esistere alcuna MdT che decida REACH. Quindi il problema di stabilire se M entra mai in uno stato q su qualche input è indecidibile. $\square$

2. Equivalenza di due MdT M e N

Problema. Dati M e N , stabilire se accettano lo stesso insieme di input.

Teorema 9.2. EQ_{TM} non è decidibile.

Dimostrazione. Faremo una riduzione dal problema

$$\text{EMPTY}_{\text{TM}} = \{ \langle M \rangle : L(M) = \emptyset \},$$

noto indecidibile.

Costruzione della riduzione. Sia N_0 la MdT che rifiuta ogni input (quindi $L(N_0) = \emptyset$). Definiamo la funzione calcolabile

$$\sigma: \langle M \rangle \mapsto \langle M, N_0 \rangle.$$

Allora, per ogni M ,

$$L(M) = \emptyset \iff L(M) = L(N_0) \iff \langle M, N_0 \rangle \in \text{EQ}_{\text{TM}}.$$

Se esistesse una MdT D che decidesse EQ_{TM} , basterebbe su input $\langle M \rangle$:

1. calcolare $\sigma(\langle M \rangle) = \langle M, N_0 \rangle$;
2. eseguire D su $\langle M, N_0 \rangle$.

Allora

$$D(\langle M, N_0 \rangle) \text{ accetta} \iff \langle M, N_0 \rangle \in \text{EQ}_{\text{TM}} \iff L(M) = \emptyset,$$

cioè D deciderebbe anche EMPTY_{TM} , assurdo.

Conclusione. Non esiste alcuna MdT che decida EQ_{TM} . Pertanto il problema di stabilire se due MdT accettano lo stesso linguaggio è indecidibile. $\square$

3. L'insieme $T = \{ M : M \text{ si ferma su ogni input} \}$ e il suo complemento

Teorema 9.3. Sia $T = \{ M : M \text{ si arresta su ogni input} \}$. Allora né T né il suo complementare $\bar{T}$ sono decidibili.

Sia

$$T = \{ \langle M \rangle : M \text{ si arresta su ogni input} \}, \quad \bar{T} = \{ \langle M \rangle : \exists x, M \text{ non si arresta su } x \}.$$

Di seguito mostriamo che né T né $\bar{T}$ sono decidibili.

Teorema 9.4. Il linguaggio T non è decidibile e nemmeno il suo complementare $\bar{T}$ è decidibile.

Dimostrazione. **(1) T non è decidibile.** Supponiamo, per assurdo, che esista una MdT D_T che decide T . Dobbiamo allora decidere anche il problema dell'Halting

$$\text{HALT} = \{ \langle M, x \rangle : M \text{ si arresta su } x \},$$

indecidibile. Costruiamo una trasformazione computabile

$$\langle M, x \rangle \mapsto \langle M_x \rangle$$

dove M_x è la macchina che, su input y ,

$$M_x(y) = \begin{cases} \text{simula } M \text{ su } x \text{ e si arresta se e solo se } M(x) \text{ si arresta,} & \text{se } y = x, \\ \text{si arresta subito,} & \text{se } y \neq x. \end{cases}$$

Allora:

$$M(x) \text{ si arresta} \iff M_x \text{ si arresta su ogni input} \iff \langle M_x \rangle \in T.$$

Di conseguenza, se D_T decidesse T , basterebbe inviarle $\langle M_x \rangle$ per decidere HALT, assurdo. Quindi T non è decidibile.

(2) $\bar{T}$ non è decidibile. Allo stesso modo riduciamo il complemento di HALT, $\overline{\text{HALT}} = \{\langle M, x \rangle : M \text{ non si arresta su } x\}$, a $\bar{T}$. Usiamo la stessa costruzione di M_x sopra:

$$M(x) \text{ non si arresta} \iff M_x \text{ non si arresta su } x \iff \langle M_x \rangle \in \bar{T}.$$

Se esistesse una MdT $D_{\bar{T}}$ che decidesse $\bar{T}$, potremmo decidere $\overline{\text{HALT}}$, il che è impossibile. Quindi $\bar{T}$ non è decidibile né questo né il suo complementare. □

10 Esercizio 10

Enunciato

Abbiamo a disposizione (ipoteticamente) un “oracolo” che risolve il problema della fermata (*Halting Problem*) per una macchina di Turing M e un input x . Dobbiamo mostrare che, grazie a tale oracolo, potremmo decidere la veridicità sia dell’Ultimo Teorema di Fermat (UTF) sia della Congettura di Goldbach (CG).

Idea generale

Entrambe le affermazioni (UTF e CG) sono del tipo: “Per ogni n (o per ogni numero maggiore di 2) vale una certa proprietà...”. Se avessimo la possibilità di risolvere *sempre* il problema della fermata, potremmo codificare un programma che, in maniera potenzialmente infinita, verifica tutti i casi possibili (di UTF o di CG) e si ferma se trova un controesempio. Con l’oracolo dell’Halting Problem, potremmo “spezzare” l’eventuale esecuzione infinita e distinguere fra due casi:

1. ****Non esiste controesempio**** (cioè la congettura è vera): la macchina non si fermerà mai (non trovando alcun test che fallisca).
2. ****Esiste un controesempio****: la macchina lo trova a qualche passo k e si ferma.

Costruzione di un ipotetico programma risolutore

Per l’Ultimo Teorema di Fermat:

1. Progettiamo una MdT M_{Fermat} che genera in sequenza *tutti* i possibili triple (x, y, z) con $x, y, z \in \mathbb{N}^+$ e $n > 2$ (o equivalenti varianti di enumerazione).
2. Per ogni tripla, testa se $x^n + y^n = z^n$. - Se trova una soluzione (cioè un *controesempio* all’UTF), si ferma. - Altrimenti va avanti con la successiva tripla.

Se l’UTF è *falso*, esiste un controesempio e quindi M_{Fermat} si fermerà. Se invece l’UTF è vero, M_{Fermat} non troverà mai una soluzione e non si fermerà.

Per la Congettura di Goldbach:

1. Progettiamo una MdT M_{Goldbach} che enumera tutti i numeri pari $n > 2$ (ad esempio 4, 6, 8, ...).
2. Per ciascuno, controlla se si può scrivere $n = p + q$ con p, q primi. Se non lo si riesce a fare, si ferma immediatamente (ha trovato un controesempio alla congettura). Altrimenti prosegue col prossimo n .

Se esiste un numero pari (maggiore di 2) che *non* è somma di due primi, M_{Goldbach} lo troverà e si fermerà. Altrimenti continuerà all'infinito.

Uso dell'oracolo per l'Halting Problem

Una volta definita la macchina M_{Fermat} (rispettivamente M_{Goldbach}), possiamo costruire un algoritmo R_{Fermat} che:

- Prende in input la descrizione di M_{Fermat} e un input fittizio (ad es. vuoto).
- Interroga l'oracolo dell'Halting Problem: " M_{Fermat} si ferma su input vuoto?"
- Se l'oracolo risponde *Sì*, significa che M_{Fermat} trova un controesempio all'UTF; dunque l'UTF è falso.
- Se l'oracolo risponde *No*, significa che M_{Fermat} non si ferma mai, ossia *non* esistono controesempi, e dunque l'UTF è vero.

Un identico schema si applica a M_{Goldbach} per risolvere la CG.

Conclusione

Se avessimo (per assurdo) un modo per risolvere sempre il problema della fermata, potremmo verificare *decisamente* (cioè in tempo finito) la veridicità o falsità di enunciati matematici estremamente profondi, come l'UTF o la CG, semplicemente programmando macchine che cercano controesempi in modo esaustivo e chiedendo all'oracolo dell'Halting Problem se tali macchine si fermeranno mai.

Ovviamente, questo mostra quanto "potente" e "irrealistico" sarebbe un oracolo per l'Halting Problem (date le attuali conoscenze): risolverebbe problemi che sappiamo richiedere dimostrazioni matematiche sofisticate o tuttora ignote.